Shared Brain Cell
Technical Documentation

Operating Systems Final Project
Version 1.0

Group: Shared Brain Cells
John Rence Arenga, Kian Chris Po, Cj Sajonia,
Neil Sermeno, Kent Cedric Pacios

1. Introduction

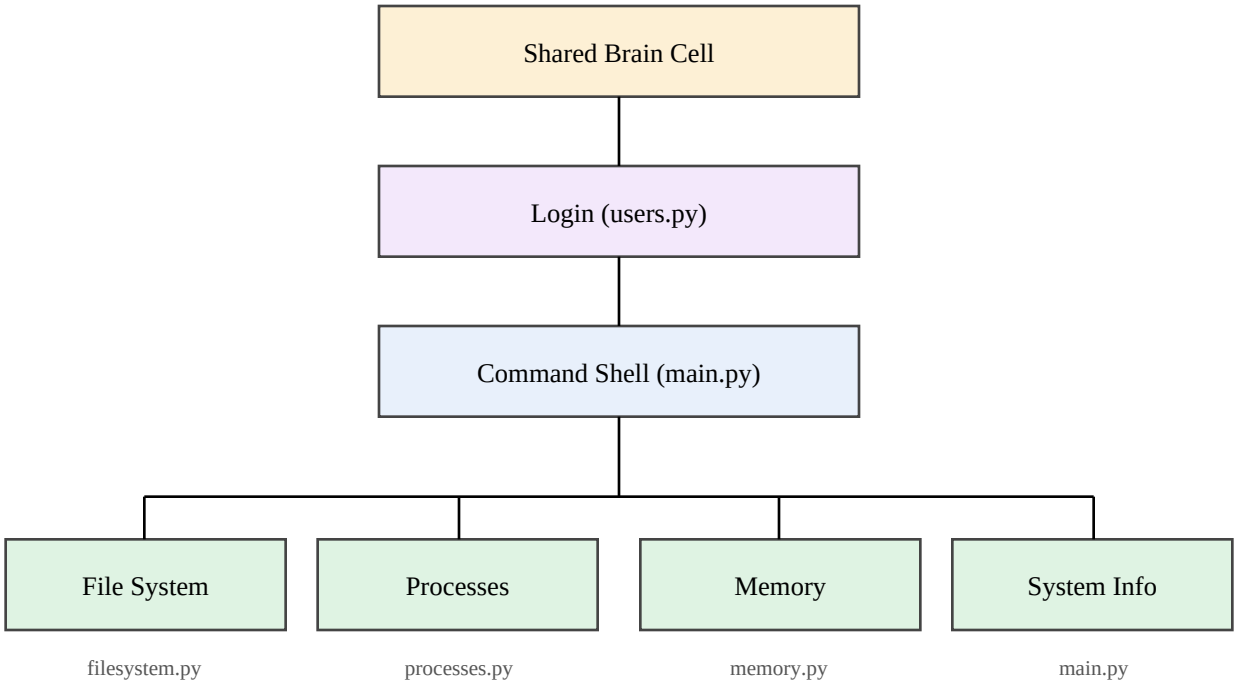
Shared Brain Cell is a command-line operating system simulator written in Python. It runs in Windows Command Prompt. It does not boot a real system and does not control real hardware. It shows how a shell, a file system, processes, memory, and users work together in an operating system.

2. Project Objectives

- Build a working command-line shell with its own prompt
- Simulate a file system with folders and files
- Simulate process creation and termination with PIDs
- Simulate memory allocation and release
- Add a login system with user accounts
- Handle errors without crashing
- Document and test the system, and host it on GitHub

3. System Architecture

The shell in main.py reads each command and sends it to the correct module. The login runs first. The shell starts only after a good login.



4. Program Structure

File	Purpose
main.py	Starts the system, runs the login, shows the prompt, reads each command, and calls the right module. Holds help, clear, date, time, whoami, systeminfo, and exit.
users.py	Stores the user accounts and runs the login with three tries.
filesystem.py	The FileSystem class. Handles dir, mkdir, create, write, read, delete, and cd.
processes.py	The ProcessManager class. Handles run, process, and kill.
memory.py	The MemoryManager class. Handles memory, allocate, and free.

5. File Management

The file system lives in the computer's memory. A folder is a Python dictionary. A file is a text string stored under its name inside a folder. A list called path remembers which folder is open.

```
root = {
  "Documents": {
     "notes.txt": "hello"
  }
}
path = ["Documents"]   # current folder
```

- mkdir adds an empty dictionary under a name

- create adds an empty string under a name

- write replaces the string of an existing file

- read returns the string of a file

- delete removes a name. A folder must be empty first

- cd adds a folder name to path, and cd .. removes the last name

- dir lists every name in the current folder and marks folders with DIR

Files reset when the program closes because nothing is written to the disk.

6. Process Management

A process is a dictionary entry. The key is the PID. The value holds the name and the status. A counter gives each new process the next PID, starting from 1. PIDs are never reused in one session.

```
table = {
  1: {"name": "calculator", "status": "RUNNING"},
  2: {"name": "editor",     "status": "RUNNING"}
}
```

- run creates an entry and prints its PID, name, and status

- process prints every entry in a table

- kill removes the entry. An unknown PID gives an error

These are only simulated. No real Windows process starts or stops.

7. Memory Management

The system has 1024 MB of simulated memory. It starts with 256 MB used, like a system reserve. The class keeps two numbers: total and used. Free memory is total minus used.

- allocate adds an amount to used. It fails if the amount is more than the free memory

- free subtracts an amount from used. It fails if the amount is more than the used memory

- memory prints total, used, and free

- Amounts of zero or less are refused

This does not touch the real RAM.

8. User Management

users.py holds a dictionary of usernames and passwords. Two accounts exist: admin and student. At startup the program asks for a username and a password. The password is hidden with the getpass module. After three wrong tries the program closes. After a good login the username is kept in a variable. The whoami and systeminfo commands read it.

Username	Password
admin	admin123
student	student123

9. Error Handling

Every command checks its input before it acts. The program prints a clear message and goes back to the prompt. It never closes on bad input. The main loop also catches Ctrl+C and end of input and shuts down in a clean way.

Situation	Message
Unknown command	Unknown command. Type 'help' to see the available commands.
Missing file	Error: File does not exist.
Unknown PID	Error: Process does not exist.
PID or amount is not a number	Error: Please enter a whole number.
Missing argument	Error: Missing argument. Usage: ...
Name already used	Error: Name already exists.
Too much memory requested	Error: Not enough free memory.
Too much memory released	Error: Cannot free more than the used memory.
Folder not empty	Error: Directory is not empty.
Wrong login	Error: Wrong username or password.

10. Testing

We ran every command and checked the output against the assignment. We also entered wrong input to check the error messages.

Test	Expected result	Result
Login with admin and admin123	Login successful	Pass
Login with a wrong password	Error message and a new try	Pass
help	Shows the command list	Pass
mkdir, create, write, read, dir	Folder and file made, text read back	Pass
cd and cd ..	Prompt shows the folder and then returns	Pass
delete on a full folder	Error: Directory is not empty	Pass
delete on a file and an empty folder	File deleted	Pass
run, process, kill 1	Process starts, is listed, and ends	Pass
kill 999 and kill abc	Error messages	Pass
allocate 200, memory, free 200	Used memory goes up and back down	Pass
allocate 5000 and free 5000	Error messages	Pass
whoami, date, time, systeminfo	Correct information	Pass
hellooo	Unknown command message	Pass
clear and exit	Screen clears, shutdown message	Pass

11. Problems Encountered

Problem	Solution
Git showed a machine type mismatch and refused to run.	We checked the System type in Windows Settings, uninstalled Git, and installed the version for our processor.
The first commit failed because Git did not know our identity.	We set user.name and user.email with git config, then committed again.
The push failed with the message src refspec main does not match any.	The failed commit left nothing to push. After the commit worked, the push worked.
Windows did not know the python command.	We reinstalled Python with Add Python to PATH ticked, or used the py command.
Group members edited the same files.	Each member owns one file. Everyone runs git pull before they start work.

12. Conclusion

This project taught us how the main parts of an operating system connect. The shell takes a command and sends it to the file system, the process table, or the memory manager. We learned that a PID identifies a process, that memory needs limits and checks, and that a good system handles bad input. We also learned how to work as a group with GitHub. Our system is a simulation, but it follows the same ideas a real operating system uses.